MolPredictor_Alpha 实验报告

一、实验目的

本实验旨在应用并巩固机器学习基础知识，为后续深度学习实践奠定基础。具体目标如下：

1.掌握 Tox21 数据集的加载、预处理与特征工程方法，完成数据划分。

2.实现并比较多种机器学习算法在化合物毒性分类任务上的性能，包括：KNN、逻辑回归（含 L1、L2 正则化）、随机森林以及 XGBoost。

3.观察正则化（L1/L2）对逻辑回归模型的影响；对比 KNN、随机森林与 XGBoost 在不同数据划分下的表现差异，体会线性模型与树模型在高维稀疏且类别不平衡数据上的适用性。

4.探索 XGBoost 在极度不平衡分类中的调参策略（包括类别权重调整、阈值移动等），尝试提升毒性样本的召回率。

将所学的机器学习理论应用于可运行的工程代码实现，完成从数据到评估的完整流程。

二、实验原理

2.1 tox21 数据集与分子指纹

tox21 数据集是美国 Tox21 项目发布的化合物毒性筛选数据。每个化合物由 SMILES 字符串表示，经过 Morgan 指纹（1024 位） 编码为二值特征向量。本实验选用 NR-AR（雄激素受体通路）毒性任务作为二分类目标（1：有毒，0：无毒）。

2.2 K 近邻算法（KNN）

KNN 是一种基于距离的懒惰学习算法。对于测试样本，算法在训练集中找到与其最近的 K 个样本，根据这些样本的类别进行投票（或加权投票）决定预测结果。

本实验中设置 n_neighbors=（3,4,5,6,7,8），权重方式为有 weights='distance'（距离越近的样本权重越大）或默认无权重，距离度量采用欧氏距离。KNN 无需显式训练，但预测时需计算所有样本距离，复杂度较高。

2.3 逻辑回归

逻辑回归是一种线性分类模型，通过 Sigmoid 函数将线性组合输出映射到 (0,1) 区间，表示样本属于正类的概率：

$$P(y=1 \mid x) = \frac{1}{1+e^{-(w^T x+b)}}$$

模型采用 交叉熵损失，通过梯度下降优化。

2.4 正则化（L1 与 L2）

为防止过拟合，常在损失函数中加入参数惩罚项：

· L2 正则化（Ridge）：添加 $\lambda\|w\|_2^2$ ，使权重趋向于较小值但通常不为零，提升泛化能力。

· L1 正则化（Lasso）：添加 $\lambda\|w\|_1$，可产生稀疏解（部分权重精确为零），实现特征选择。

本实验对比了三种逻辑回归：

· 默认 L2：solver='liblinear'，C=1.0（C 为正则化强度倒数，C 越小正则化越强）

· 显式 L2：solver='lbfgs'，C=0.0001，class_weight='balanced'

· L1：solver='saga'（支持 L1），l1_ratio=1.0，C=0.05，class_weight='balanced'

注：class_weight='balanced' 自动调整类别权重，缓解类别不平衡问题。

2.5 随机森林

随机森林是一种基于 Bagging 的集成学习方法。它通过 Bootstrap 采样构建多棵决策树，并在每棵树的节点分裂时随机选择部分特征子集，最终通过投票（分类）或平均（回归）输出结果。随机森林能够处理高维数据，对噪声和过拟合具有较好的鲁棒性，且能捕捉特征之间的非线性交互关系。本实验通过手动与自动调参（max_depth、min_samples_leaf 等）优化模型性能。

2.6 XGBoost

XGBoost（eXtreme Gradient Boosting）是一种基于 梯度提升框架 的集成学习算法。它通过迭代地添加决策树，每棵新树拟合前一棵树的残差（负梯度），并利用二阶导数信息加速优化。XGBoost 引入了多种正则化手段（如树的叶子节点权重惩罚、列采样等）防止过拟合，同时支持自定义损失函数与评估指标。

针对本任务中极度不平衡的正负样本（约 1:22），XGBoost 提供了 scale_pos_weight 参数来调整正类的权重，以提升少数类（毒性分子）的召回率。此外，阈值移动（Threshold Moving） 是一种后处理技术：通过降低分类决策阈值（如从 0.5 降至 0.3），可使模型对正类更敏感，从而提高召回率，但可能增加虚警率。

2.7 评价指标

ROC-AUC：衡量模型区分正负类的能力，不受阈值影响。

混淆矩阵：展示 TP、TN、FP、FN。

精确率（Precision）、召回率（Recall）、F1 值：综合评估分类性能，尤其关注正类（有毒化合物）的预测效果。在不平衡分类中，召回率比准确率更具参考价值。

三、实验环境

3.1 硬件环境

项目	配置
CPU	AMD Ryzen 9 8945HX with Radeon Graphics, 16 核 32 线程，基准频率 2.50GHz
内存	32GB（两条 16GB DDR5 5200 MHz，制造商 CXMT）
硬盘	两块 945GBPCI-E SSD（忆联 UMIS 系列）
GPU	集成：AMD Radeon 610M(512MB) 独立：NVIDIA GeForce RTX 5060 Laptop GPU(8GB)

3.2 软件环境

项目	版本
操作系统	Windows 11 家庭中文版 + WSL2(Ubuntu 24.04.1 LTS)
Python	3.12.3
numpy	2.4.4
pandas	3.0.2
scikit-learn	1.8.0
matplotlib	3.10.8

seaborn	0.13.2
rdkit	2025.9.6
xgboost	3.2.0
其他	Joblib 1.5.3,scipy 1.17.1, threadpoolctl 3.6.0 等

四、实验步骤

4.1 KNN 算法

实验组别	参数设置	ROC-AUC	准确率(ACC)	F1-Score (类别 1)	精确率 (Pre 类别 1)	召回率 (Rec 类别 1)	混淆矩阵 (TN/FP/FN/TP)
第一组	K=5	0.7579	0.97	0.58	0.87	0.44	1387/4/35/27
第二组	K=7	0.7556	0.97	0.58	0.93	0.42	1389/2/36/26
第三组	K=3	0.7323	0.97	0.57	0.90	0.42	1388/3/36/26
第四组	K=4	0.7522	0.97	0.57	0.90	0.42	1388/3/36/26
第五组	K=6	0.7525	0.97	0.57	0.90	0.42	1388/3/36/26
第六组	K=8	0.7456	0.97	0.58	0.93	0.42	1389/2/36/26
---	---	---	---	---	---	---	---
第七组	K=7（新 RS）	0.7171	0.97	0.53	0.83	0.39	1386/5/38/24
第八组	K=5（新 RS）	0.7203	0.97	0.52	0.77	0.39	1384/7/38/24
第九组	K=6（新 RS）	0.7231	0.97	0.53	0.83	0.39	1386/5/38/24
第十组	K=4（新 RS）	0.7257	0.97	0.53	0.76	0.40	1383/8/37/25

表 4.1 KNN 实验数据对比

实验操作:

从 K=3 到 8 调试 n_neighbors，其余参数固定为：test_size=0.2，random_state=0，stratify=y_final，weights='distance'，n_jobs=-1。记录每组实验的 ROC-AUC、准确率、F1-Score、精确率、召回率及混淆矩阵，结果汇总于表 4.1。

我的思路:

依据不同 K 值比较 ROC-AUC 的大小。在 random_state=0 的前六组测算中发现，K=3 与 K=8 的 ROC-AUC 分别为 0.7323 和 0.7456，而 K=4~7 的 ROC-AUC 均在 0.75 以上。因此切换 random_state=42 时，只测算了 K=4,5,6,7。

实验结果与初步观察:

切换随机种子后，ROC-AUC 明显下降（例如 K=5 从 0.7579 降至 0.7203）。即使在 random_state=0 的最佳情况下，KNN 的 ROC-AUC 也未能超过 0.76。

分析：这表明 KNN 模型对数据划分方式较为敏感，且在高维稀疏的分子指纹数据上存在性能瓶颈。

4.2 逻辑回归

```
Project MolPredictor_Alpha: KNN Report
Task: NR-AR
==============================
ROC-AUC score: 0.7556

Confusion matrix:
[[1389    2]
 [  36   26]]

Detailed metrics:
              precision    recall  f1-score   support

         0.0       0.97      1.00      0.99      1391
         1.0       0.93      0.42      0.58        62

    accuracy                           0.97      1453
   macro avg       0.95      0.71      0.78      1453
weighted avg       0.97      0.97      0.97      1453

Converting SMILES to Morgan Fingerprints...
X shape (features): (7831, 1024)
y shape (labels): (7831, 12)
```

图 4.2.1 KNN 第二组实验原始数据

```
==============================
Project MolPredictor_Alpha: Logistic Regression Report
Task: NR-AR
==============================
ROC-AUC Score: 0.7344
[[1342   49]
 [  34   28]]

--- Detailed Metrics ---
              precision    recall  f1-score   support

         0.0       0.98      0.96      0.97      1391
         1.0       0.36      0.45      0.40        62

    accuracy                           0.94      1453
   macro avg       0.67      0.71      0.69      1453
weighted avg       0.95      0.94      0.95      1453


--- Top 5 Toxic Structural Fragments (Indices) ---
Fingerprint Bit 646: Weight 0.7725
Fingerprint Bit 429: Weight 0.7443
Fingerprint Bit 799: Weight 0.6267
Fingerprint Bit 893: Weight 0.6193
Fingerprint Bit 717: Weight 0.5897
```

图 4.2.2 逻辑回归原始数据

实验操作:

训练默认逻辑回归模型（solver='liblinear'，采用默认的 L2 正则化，C = 1.0），采用与 KNN 相同的数据划分和标准化预处理。

我的思路:

初步测试发现，采用默认 L2 正则化的逻辑回归仍出现过拟合现象（训练集 AUC 高而测试集 AUC 较低），其测试集性能不如 KNN 的最佳结果。具体对比见图 4.2.1（KNN 第二组）和图 4.2.2（逻辑回归默认）。

分析：线性模型在该任务上若不加以正则化，容易受高维特征和噪声影响，泛化能力不足。

4.3 逻辑回归（L1L2 正则化）

实验编号	C 参数	类别权重	ROC-AUC	F1-Score（1.0）	Recall（1.0）	有效特征数	备注
1	0.5	None	0.7501	0.41	0.37	651	基准模型
2	0.5	Balanced	0.7411	0.33	0.48	731	引入权平衡，Recall 提升
3	0.1	Balanced	0.7445	0.31	0.48	586	开始压缩特征
4	0.05	Balanced	0.7443	0.34	0.53	1024	特征数异常

实验编号	C 参数	类别权重	ROC-AUC	F1-Score（1.0）	Recall（1.0）	有效特征数	备注
5	0.01	Balanced	0.7369	0.29	0.48	137	显著压缩
6	0.001	Balanced	0.7147	0.44	0.45	3	极度压缩
7	0.0001	Balanced	0.5	0	0	0	模型失效（欠拟合）

表 4.3.1 L1 正则化

注：C=0.05 时未产生稀疏性，所有 1024 个特征权重均非零，可能因正则化强度尚不足以触发精确零截断。

实验编号	C 参数	ROC-AUC	Accuracy	F1-score (1.0)	Recall (1.0)	Precision (1.0)	备注
Exp 1	0.5	0.7325	0.91	0.29	0.4	0.22	基准模型（L2）
Exp 2	0.1	0.7418	0.92	0.33	0.47	0.26	性能有所提升
Exp 3	0.05	0.7453	0.92	0.32	0.47	0.25	性能趋于稳定
Exp 4	0.01	0.7476	0.92	0.34	0.47	0.26	AUC 最高点
Exp 5	0.001	0.7474	0.93	0.36	0.48	0.29	综合指标均衡
Exp 6	0.0001	0.7485	0.94	0.39	0.47	0.34	F1/Precision 最高

表 4.3.2 L2 正则化

维度	L1 正则化（Sparsity 导向）	L2 正则化（Stability 导向）
最高 ROC-AUC	0.7501 (C=0.5)	0.7485 (C=0.0001)
最高 F1-score	0.44 (C=0.001)	0.39 (C=0.0001)
特征处理方式	特征选择：将不重要的权重直接归零	权重衰减：缩小权重但保留所有特征
有效特征数	随 C 减小剧烈下降（651 → 3 → 0）	始终保持 1024 个有效特征
模型鲁棒性	对参数极其敏感，过强的正则化会导致失效	非常稳健，即使 C 极小仍有预测力
AIDD 物理意义	提取核心毒性碎片（Key Fragments）	综合考虑全部分子的结构特征

表 4.3.3 L1L2 正则化数据对比

4.3.1 逻辑回归（L1 正则化）

实验操作:

采用 solver='saga', l1_ratio=1.0, 测试不同 C 值（0.5, 0.1, 0.05, 0.01, 0.001, 0.0001）及类别权重（class_weight=None 或'balanced'）。记录 ROC-AUC、F1-Score、召回率及有效特征数，见表 4.3.1。

我的思路:

第一次实验（C=0.5，无 class_weight）作为基准。第二次实验增加 class_weight='balanced'（后续实验均使用该设置）。第三次到第七次逐步减小 C 值，观察特征压缩效果。

分析：随着 C 减小（正则化增强），有效特征数从 651 逐步下降至 3（C=0.001），C=0.0001 退化为随机分类器（AUC≈0.5）。说明 L1 能够实现特征选择，但正则化强度需谨慎控制。

4.3.2 逻辑回归（L2 正则化）

实验操作:

采用 solver='lbfgs'，l1_ratio=0，测试 C=0.5, 0.1, 0.05, 0.01, 0.001, 0.0001，均使用 class_weight='balanced'。记录指标见表 4.3.2。

我的思路:

与 L1 不同，L2 不会将权重直接置零，而是平滑地压制所有特征的权重大小，以防止模型过度依赖单一分子片段。表 4.3.3 对比了两种正则化的效果。

分析：实验表明，在 Tox21 这种稀疏且正负样本极度不平衡（约 1:22）的数据集中，线性分类边界（逻辑回归）

极其容易陷入“高准确率、低召回率”的困境。即便使用了 class_weight='balanced'，模型在面对复杂的分子空间结构时仍达到性能瓶颈。

4.4 随机森林

阶段	编号	max_depth	min_samples_leaf	Train AUC	Test AUC	Gap(过拟合)	F1 (1.0)	Precision	Recall
I. 广度探索	1	15	1	0.9975	0.7672	0.2303	0.58	0.93	0.42
	2	8	5	0.9428	0.8	0.1428	0.52	0.64	0.44
	3	8	6	0.9367	0.793	0.1437	0.52	0.64	0.44
	4	8	7	0.9305	0.7918	0.1386	0.5	0.59	0.44
	5	8	4	0.9512	0.8079	0.1433	0.53	0.69	0.44
	6	7	4	0.936	0.7933	0.1427	0.52	0.64	0.44
	7	7	5	0.9304	0.7864	0.1439	0.51	0.6	0.45
	8	7	6	0.9238	0.795	0.1288	0.49	0.53	0.45
	9	7	7	0.9172	0.786	0.1312	0.47	0.48	0.45
II. 中深精调	10	9	4	0.9603	0.8016	0.1587	0.54	0.71	0.44
	11	9	5	0.9541	0.7999	0.1542	0.53	0.68	0.44
	12	9	6	0.9485	0.8021	0.1464	0.52	0.64	0.44
	13	9	7	0.9419	0.7889	0.153	0.5	0.6	0.44
	14	7	3	0.9416	0.7936	0.148	0.51	0.61	0.44
	15	8	3	0.9575	0.7925	0.165	0.52	0.66	0.44
	16	9	3	0.9669	0.8035	0.1634	0.53	0.69	0.44
III. 高深边界	17	10	3	0.9754	0.8048	0.1706	0.53	0.68	0.44
	18	10	4	0.9678	0.7946	0.1732	0.53	0.69	0.44
	19	10	5	0.962	0.7831	0.1789	0.5	0.6	0.44
	20	10	6	0.9559	0.7926	0.1633	0.51	0.63	0.44
	21	10	7	0.9507	0.7937	0.157	0.5	0.6	0.44

表 4.4.1 手动调参数据对比

```
--- Start automatic hyperparameter tuning (based on 5-fold CV on the training set)---
Optimal parameters balancing both AUC and Gap: {'max_depth': 1, 'min_samples_leaf': 1, 'n_estimators': 200}

--- Overfitting Check ---
Train AUC: 0.7855
Test AUC: 0.7639
Gap: 0.0216

==============================
Project MolPredictor_Alpha: Random Forest Report(自动化调参结果)
Task: NR-AR
==============================
ROC-AUC Score: 0.7639

Confusion Matrix:
[[1287  104]
 [  33   29]]

Detailed Metrics:
              precision    recall  f1-score   support

         0.0       0.97      0.93      0.95      1391
         1.0       0.22      0.47      0.30        62

    accuracy                           0.91      1453
   macro avg       0.60      0.70      0.62      1453
weighted avg       0.94      0.91      0.92      1453


--- Top 5 Important Bits (RF Gini Importance) ---
Bit 36: 0.0550
Bit 973: 0.0500
Bit 714: 0.0450
Bit 519: 0.0450
Bit 1019: 0.0400
```

图 4.4.2 第一次自动化调参原始数据

```
--- Start automatic hyperparameter tuning (based on 5-fold CV on the training set)---
Optimal parameters balancing both AUC and Gap: {'max_depth': 5, 'min_samples_leaf': 8, 'n_estimators': 1000}

--- Overfitting Check ---
Train AUC: 0.8738
Test AUC: 0.7797
Gap: 0.0941

==============================
Project MolPredictor_Alpha: Random Forest Report(Automatic hyperparameter tuning results)
Task: NR-AR
==============================
ROC-AUC Score: 0.7797

Confusion Matrix:
[[1353   38]
 [  34   28]]

Detailed Metrics:
              precision    recall  f1-score   support

         0.0       0.98      0.97      0.97      1391
         1.0       0.42      0.45      0.44        62

    accuracy                           0.95      1453
   macro avg       0.70      0.71      0.71      1453
weighted avg       0.95      0.95      0.95      1453


--- Top 5 Important Bits (RF Gini Importance) ---
Bit 519: 0.0585
Bit 36: 0.0533
Bit 84: 0.0470
Bit 250: 0.0443
Bit 138: 0.0378
```

图 4.4.3 第二次自动化调参原始数据

实验操作:

采用基于 Bagging 的随机森林算法，通过手动调参与自动调参优化模型。手动调参改变 max_depth 和 min_samples_leaf，记录训练集 AUC、测试集 AUC、过拟合 Gap（训练 AUC - 测试 AUC）及召回率等，见表 4.4.1。自动调参使用网格搜索，结果见图 4.4.2 和图 4.4.3。

我的思路:

树模型能够捕捉特征之间的非线性交互关系，有望突破线性模型的瓶颈。手动调参阶段，通过调整 max_depth 和 min_samples_leaf 来控制过拟合 Gap，同时观察 AUC 上限。因计算资源有限，手动调参估计大致范围后，再采用自动化调参。

第一次自动化调参发现模型将 Gap 压得很小（几乎为 0），但牺牲了测试集 AUC。随后增加惩罚系数、改变评分策略，将 Gap 限制在 0.1 附近，再观察最大 AUC。

分析：从表 4.4.1 可见，在不过度过拟合的情况下（Gap≈0.14~0.17），测试集 AUC 最高可达 0.8079（编号 5，max_depth=8, min_samples_leaf=4），均高于此前所有线性模型，证明非线性分类器在 Tox21 数据集上优于线性分类器。但召回率始终未能突破 0.45，因此需要尝试更强的集成模型 XGBoost。

4.5 XGBoost

实验阶段	核心策略描述	Train AUC	Test AUC	AUC Gap	Recall（正类）	Precision	混淆矩阵（TN/FP/FN/TP）	核心诊断与结论
阶段一	常规超参数搜索(默认阈值 0.5)	0.9026	0.7930	0.1095	0.44	0.69	1379/12/35/27	泛化极佳，误报极低，但陷入“高精确、低召回”陷阱，漏报严重。
阶段二	极端类别权重惩罚(引入极端 scale_pos_weight)	0.9612	0.7547	0.2066	0.55	0.11	1120/271/28/34	打破 0.45 召回瓶颈，但导致模型严重过拟合(Gap 飙升)且“草木皆兵”
阶段三	稳健权重+阈值移动(理性权重+预测阈值降至 0.3)	0.9631	0.7630	0.2001	0.63	0.08	958/433/23/39	最佳应用策略。容忍一定程度的假阳性与过拟合。换取早期虚拟筛选中极具价值的高毒性捕获率。

表 4.5 XGBoost 实验数据

实验操作:

采用 XGBoost 算法，通过三阶段网格搜索（RandomizedSearchCV）调整超参数，包括 scale_pos_weight、gamma、reg_lambda 等。记录训练集 AUC、测试集 AUC、召回率、混淆矩阵，见表 4.5。

我的思路:

为了打破 0.45 的召回率瓶颈，引入基于二阶梯度提升框架的 XGBoost。

· **阶段一（常规超参数搜索，默认阈值 0.5）**：模型测试 AUC 达到 0.7930，Gap 仅 0.1095，泛化能力优秀，但召回率仍为 0.44。

分析：在极度不平衡（1:22）的数据下，XGBoost 为追求整体准确率而变得极度保守，漏报了大量毒性分子。

· **阶段二（极端类别权重惩罚）**：将 scale_pos_weight 提高到极端值（约 10 倍），强制模型关注少数类。召回率提升至 0.55，但训练 AUC 飙升至 0.9612，Gap 扩大至 0.2066，并产生 271 个假阳性。

分析：模型严重过拟合，失去了工业泛化价值，但证明了权重调整对召回率的正向作用。

· **阶段三（稳健权重训练 + 预测阈值下调）**：采用"训练与预测解耦"策略。

训练端：回调 scale_pos_weight 至合理范围，配合 gamma=0.1、reg_lambda=5 进行正则化约束，确保模型学到真实的化学规律。

预测端：引入阈值移动（Threshold Moving），将分类阈值从 0.5 降至 0.3（即只要有 30% 的毒性概率即报警），以适应早期毒性筛查中"漏报代价极大"的场景。

分析：阶段三在保持测试 AUC 约 0.7630 的同时，召回率突破至 0.63（成功拦截 62 个毒性样本中的 39 个），虽然假阳性增加（433 个），但实现了高灵敏度的早期毒性预警。

五、实验结果与讨论

5.1 各模型最佳性能综合对比

将各模型在实验中获得的最佳性能指标汇总于表 5.1

模型类别	最优策略/超参数	Test AUC	Precision	Recall	F1-Score	漏报数 (FN)	误报数 (FP)	核心应用评价
K 近邻 (KNN)	K=5, distance	0.7579	0.87	0.44	0.58	35	4	维数灾难导致泛化差，且对数据划分极度敏感
逻辑回归 (L1)	C=0.5，无类别平衡	0.7501	0.46	0.37	0.41	39	27	漏报最严重，但能将有效特征压缩至 651 个
逻辑回归 (L2)	C=0.0001, balanced	0.7485	0.34	0.47	0.39	33	56	引入权重平衡后漏报略减，模型鲁棒性强但精度见顶
随机森林 (RF)	max_depth=8, leaf=4	0.8079	0.69	0.44	0.53	35	12	取得了全场最高泛化 AUC，但陷入了少数类陷阱
XGBoost (泛化优先)	常规搜索调参，阈值 0.5	0.793	0.69	0.44	0.54	35	12	防过拟合表现优秀，但对毒性样本的捕获依然保守
XGBoost (召回优先)	稳健正则化 + 阈值降至 0.3	0.763	0.08	0.63	0.14	23 (全场最低)	433	打破召回瓶颈，以 433 个假阳性为代价，成功多拦截了 12 个潜在毒性分子

表 5.1 综合各模型最佳性能数据对比

5.2 分析与讨论

5.2.1 KNN 的局限性与维数灾难

实验结果表明，KNN 在固定随机种子时最佳 AUC 为 0.7579（K=5），但更换随机种子后 AUC 降至 0.72 左右，反映出模型对数据划分方式极度敏感。更本质的原因在于，Morgan 指纹作为 1024 维的二值稀疏向量，使得样本特征空间极度膨胀。在这种高维空间中，样本间的欧氏距离趋于一致（即“维数灾难”，Curse of Dimensionality），导致 KNN 无法基于距离有效计算出近邻，进而失去了类别区分能力。此外，极度不平衡的类别分布（约 1:22）使得多数类在投票机制中占据绝对主导，这也是其召回率低迷（在 62 个正样本中漏报多达 35 个）的重要原因。

5.2.2 逻辑回归正则化效果对比与特征稀疏性

线性分类器在该任务上展现了明显的性能天花板。对于 L1 正则化，随着惩罚系数 C 值的减小，有效特征数从 651 锐减至 3，并最终在 C=0.0001 时完全失效，这验证了 L1 在高维数据下的特征选择（Feature Selection）能力。值得注意的是，当 C=0.05 时，L1 正则化并未产生预期的稀疏性（有效特征数仍为 1024）。这可能是因为正则化惩罚强度处于特征尺度的临界阈值，导致绝大多数特征权重被极度压缩（趋于无限小）但尚未触发精确的零截断。这也侧面反映了纯线性模型在处理高维化学生物数据时的数值不稳定性。

相比之下，L2 正则化通过平滑的权重衰减保留了全部 1024 个特征，表现出极强的鲁棒性，即使在 C=0.0001 时仍保持基础预测力。然而，无论是 L1 还是 L2，纯线性分类边界均无法有效解析复杂的分子结构空间。正类召回率始终难以突破 0.53，即便 L2 模型达到了最佳配置，仍存在 33 个不可忽视的漏报（FN）。

5.2.3 随机森林的非线性优势与少数类陷阱

实验证明，树模型能够突破线性模型的瓶颈。随机森林通过集成非线性决策树，获得了 0.8079 的最佳测试 AUC（max_depth=8, min_samples_leaf=4），同时过拟合 Gap 控制在 0.1433 的合理范围内，展现出优秀的整体泛化能力。

然而，一个顽固的现象是：无论超参数如何微调，其对少数类（正样本）的召回率几乎始终锁定在 0.44 左右（漏报数基本保持在 35 个）。这表明，随机森林基于 Gini 纯度分裂的底层机制虽然能够有效刻画分子特征的非线性组合，但在极度不平衡的样本分布下，决策树的生长倾向于优先拟合多数类（无毒样本）以实现全局纯度的最大化。这种机制导致模型对少数类（毒性特征）的捕获依然过于保守（仅产生 12 个假阳性误报，却漏掉了 35 个真正有毒的分子），极易陷入追求表面高精确率的“少数类陷阱”。

5.2.4 XGBoost 与早期预警的阈值偏移策略

XGBoost 的引入成功打破了上述召回率瓶颈，其三阶段实验展示了模型从“追求整体精度”向“满足特定业务场景”的工程演变：阶段一（常规搜索）取得了极佳的泛化效果，但依然陷入了“高精确、低召回”的困境（FN=35，FP=12，与随机森林高度一致）。阶段二的极端类别权重（scale_pos_weight）虽然显著提高了召回率至 0.55，但也造成了严重的过拟合与极高的误报率。

在阶段三中，模型在训练端保持适度的正则化（确保学到真实的化学规律），而在预测端应用了“阈值移动（Threshold Moving）”策略，将分类决策边界从 0.5 下调至 0.3。这一操作以牺牲部分精确率为代价（假阳性 FP 增至 433 个），将召回率大幅提升至 0.63，成功将关键的漏报数（FN）压低至所有实验最低的 23 个。在早期药物虚拟筛选（Virtual Screening）管线中，高召回率意味着潜在毒性分子被拦截的概率更大，漏报（False Negative）带来的研发灾难远大于虚报（False Positive）增加的验证成本。后续完全可以通过高精度的体外实验（in vitro）或更复杂的深度学习模型进一步剔除这 400 多个假阳性，因此这种阈值偏移策略具有较高的实际工程与产业价值。

5.3 误差来源与改进方向

特征表示的内在局限性： 现阶段采用的 Morgan 指纹本质上是分子二维拓扑结构的局部哈希映射。它未包含原子的空间立体构象（3D Conformation）以及分子内部的电荷与生化分布特征。此外，定长指纹极易产生哈希碰撞（Bit Collision），导致结构不同但指纹相似的“伪阳性”或“伪阴性”。

极端类别不平衡的制约： 尽管在算法层面尝试了类别权重调整与阈值移动，并成功减少了漏报，但数据集中高达 1:22 的正负样本失衡依然是制约模型深层次学习毒性特征的根本阻碍。未来可探索代价敏感学习（Cost-sensitive Learning）或基于异常检测（Anomaly Detection）范式的无监督特征学习。

六、结论与未来展望

本实验基于 Tox21 数据集的 NR-AR 毒性分类任务，系统性评估与优化了从线性模型（KNN、逻辑回归）到树集成模型（随机森林、XGBoost）的性能表现。

结论表明： 传统线性模型受限于分类边界，在处理高维稀疏特征时容易遭遇性能瓶颈；随机森林凭借非线性集成机制获得了在本次实验所测试的数据及模型中最佳的整体泛化精度（测试集 AUC ~ 0.8079），但在不平衡样本上面临召回率天花板 ；XGBoost 结合阈值偏移策略（分类阈值 0.3）成功实现了 0.63 的高灵敏度召回，契合早期毒性筛查“宁可错杀、不可漏放”的业务需求，可作为当前工业流水线的基础基线模型 。

综上所述，传统的基于人工指纹（Fingerprints）提取的机器学习算法在平衡高泛化能力与高召回率时，已经触及了物理表征的极限。为了根本性突破特征表示的局限，本课题组的 MolPredictor 平台在下一阶段将视角从“人工特征工程”全面转向“端到端表示学习（End-to-End Representation Learning）”。拟引入图神经网络（Graph Neural Networks, GNN），直接利用分子的拓扑图结构进行高维空间的信息传递（Message Passing）与特征聚合。这不仅有望打破高维稀疏特征带来的性能天花板，也将为分子毒性预测提供更清晰的空间子结构可解释性，从而为 AI 驱动的药物发现（AIDD）奠定更坚实的算法底座。

本实验的完整代码及运行说明已开源在 GitHub：https://github.com/vincentarthurleung-rgb/MolPredictor_Alpha